



Министерство науки и высшего образования Российской
Федерации Федеральное государственное автономное
образовательное учреждение высшего образования

«Московский государственный технический
университет имени Н.Э. Баумана
(национальный исследовательский
университет)» (МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ

КАФЕДРА ПРИКЛАДНАЯ МАТЕМАТИКА И ИНФОРМАТИКА

Место титульного листа

***К КУРСОВОЙ
РАБОТЕ НА ТЕМУ:***

***База данных приложения для тестирования знаний
по теме «Формальные языки»***

Содержание

Содержание.....	1
1. Введение.....	2
2. Проектирование базы данных.....	3
2.1. Проектирование ER модели.....	4
2.2. Проектирование реляционной модели.....	6
2.3. Нормализация реляционной модели	8
3. Реализация базы данных.....	12
3.1. Физическая модель в SQL	13
3.2. Административное приложение управления базой данных.....	15
3.3. Клиентское приложение для студентов.....	16
4. Тестирование системы.....	19
5. Заключение	21
Список литературы	23
Приложение А	23
Приложение Б.....	27
Приложение В	31
Приложение Г	42

1. Введение

В современном образовательном процессе автоматизация проверки знаний студентов становится всё более востребованной. Традиционные методы тестирования, такие как письменные экзамены или устные опросы, требуют значительных временных затрат со стороны преподавателей. С другой стороны, студенты, особенно по таким сложным дисциплинам, как “Теория формальных языков”, часто сталкиваются с трудностями в освоении материала. Преподаватели отмечают, что успешное решение задач по этой дисциплине во многом зависит от развития интуиции, которую сложно сформировать без систематической практики.

Разработка системы онлайн-тестирования с бесконечным генератором задач позволит студентам самостоятельно отрабатывать навыки, развивать интуицию и готовиться к контрольным мероприятиям, а преподавателям — ускорить процесс проверки и сосредоточиться на анализе результатов. Таким образом, создание базы данных для такой системы решает актуальные задачи как для студентов, так и для преподавателей, обеспечивая эффективное и гибкое тестирование.

Целью данной курсовой работы является проектирование и реализация базы данных для системы тестирования знаний по дисциплине “Теория формальных языков”. В процессе выполнения данного проекта стояли следующие задачи:

1. Спроектировать ER и реляционную модели базы данных.
2. Реализация базы данных с использованием реляционной СУБД.
3. Написание приложения тестирования знаний.

2. Проектирование базы данных

2.1. Проектирование ER модели

В процессе проектирования базы данных была разработана ER-модель, показанная на рисунке 1.

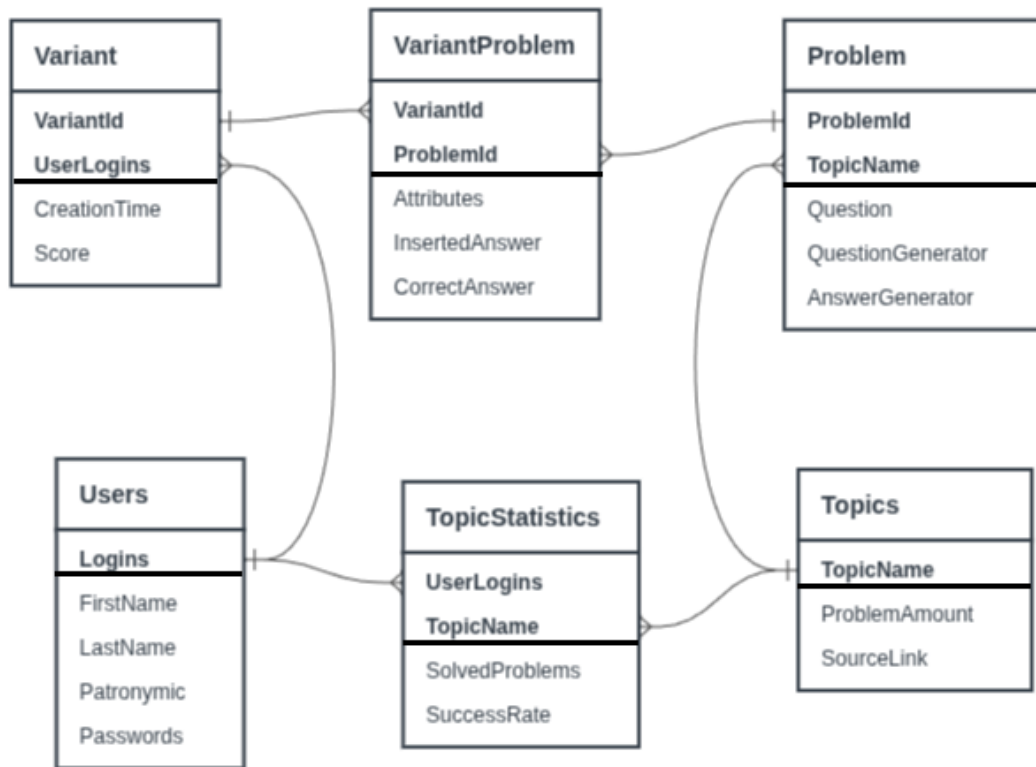


Рисунок 1 — ER модель

Модель включает в себя следующие сущности, каждая из которых выполняет определённую роль в системе:

1. **Users** — это студенты, которые будут пользоваться данным приложением. Сущность содержит атрибуты для их аутентификации (Logins, Passwords) и идентификации (FirstName, LastName, Patronymic). Эта сущность является центральной, так как связана с вариантами тестов, которые им генерируются и решаются, и статистикой студента.
2. **Topics** — это темы, в которые сгруппированы задачи. Пользователи приложения должны иметь возможность тренироваться решать

задачи одной темы, если они в ней недостаточно уверены. Атрибут `SourceLink`, если он не `NULL`, может указывать на теоретические материалы и лекции по теме, которую студент хочет изучить. `ProblemAmount` автоматически обновляется при добавлении или удалении задач, соответствуя разнообразию задач по этой теме. Дочерними сущностями для Тем являются Задачи (по этой теме) и Статистика (данного студента по данной теме).

3. `Problem` — это шаблон задачи, которую будут решать студенты. Предполагается, что именно эту сущность будут пополнять владельцы сервера. Они должны определить к какой теме относится задача, которую они придумали, расписать ее текст (или вопрос - `Question`). Чаще всего решением задачи является определенный автомат или грамматика, или регулярное выражение, поэтому автору задачи надо указать ссылку на программу, способную проверить правильность введенного студентом ответа (`AnswerGenerator`). Если придуманная задача относится к параметризуемым, и будет поддаваться генерации, надо указать ссылку на генератор параметров, подходящих для этой задачи (`QuestionGenerator`).
4. `VariantProblem` — это уже решаемая студентом задача, сгенерированная на основе шаблона, который будет считаться ее родительской сущностью. Такие сущности создаются тогда, когда в генерируемый студентом вариант теста попадает задача с ID соответствующему шаблону этой задачи. В отличие от шаблона, тут уже известны параметры (или атрибуты) конкретно этой задачи, а также правильный ответ, подсчитанный с помощью этих параметров. Правильный ответ будет показан студенту после того, как он введет свой вариант ответа и решит весь вариант, в случае если его ответ на эту задачу не является правильным.

5. Variant — это вариант теста, решаемый студентом. Эта сущность нужна для объединения задач, которые решает студент, отслеживания времени, которое понадобилось студенту на решение варианта и подсчета оценки (процент выполнения работы), которую он получит за тест.
6. TopicStatistics — это сущность, сопряжения студента и темы задачи, обновляющаяся при решении теста. Авторизованный студент может воспользоваться статистикой, чтобы выяснить, какие темы он знает хуже всего, и в чем ему следует потренироваться.

2.2. Проектирование реляционной модели

На основе ER-модели была разработана реляционная модель базы данных, показанная на рисунке 2.

www.quickdatabasediagrams.com

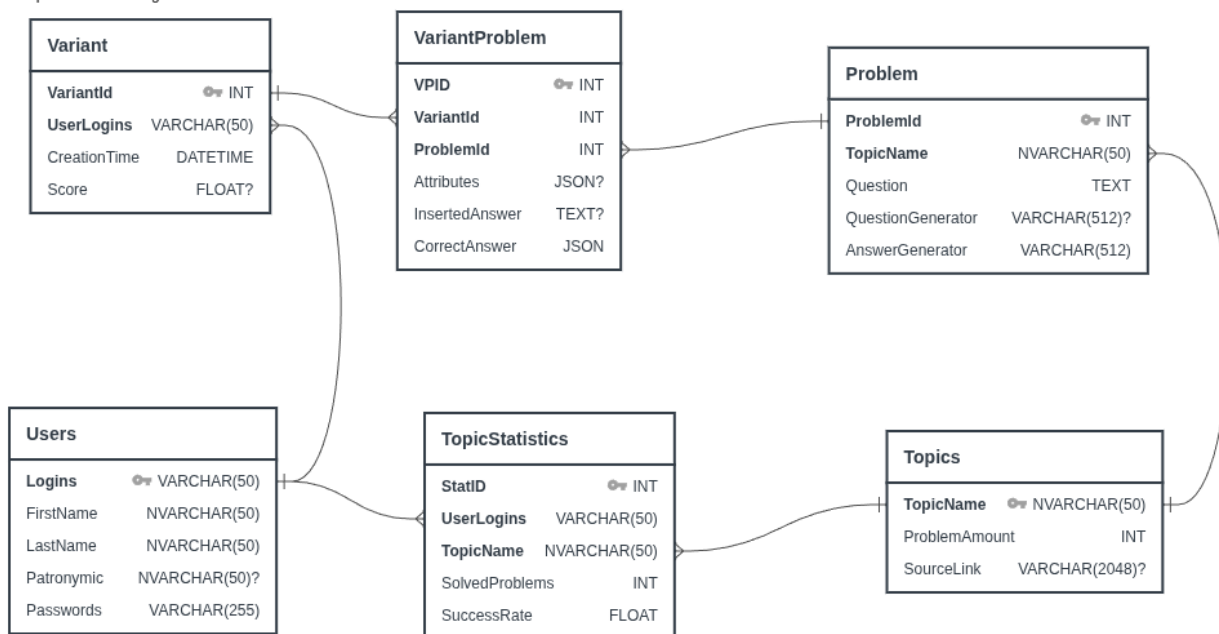


Рисунок 2 — Реляционная модель.

Таблицы для каждой сущности в этой модели представлены ниже (Таблица 1 - Таблица 6).

Таблица 1 — Users

Column Name	Type	Key	NULL Status	Remarks
Logins	VARCHAR(50)	Primary	NOT NULL	Primary key
FirstName	NVARCHAR(50)	No	NOT NULL	
LastName	NVARCHAR(50)	No	NOT NULL	
Patronymic	NVARCHAR(50)	No	NULL	
Passwords	VARCHAR(255)	No	NOT NULL	

Таблица 2 — Topics

Column Name	Type	Key	NULL Status	Remarks
TopicName	NVARCHAR(50)	Primary	NOT NULL	Primary key
ProblemAmount	INT	No	NOT NULL	Default 0
SourceLink	VARCHAR(2048)	No	NULL	

Таблица 3 — Problem

Column Name	Type	Key	NULL Status	Remarks
ProblemId	INT	Primary	NOT NULL	Surrogate key
TopicName	NVARCHAR(50)	Foreign	NOT NULL	Foreign key
Question	TEXT	No	NOT NULL	
QuestionGenerator	VARCHAR(512)	No	NULL	
AnswerGenerator	VARCHAR(512)	No	NULL	

Таблица 4 — Variant

Column Name	Type	Key	NULL Status	Remarks
VariantId	INT	Primary	NOT NULL	Surrogate key
UserLogins	VARCHAR(50)	Foreign	NOT NULL	Foreign key
CreationTime	DATETIME	No	NULL	
Score	FLOAT	No	NULL	От 0 до 1

Таблица 5 — VariantProblem

Column Name	Type	Key	NULL Status	Remarks
VPID	INT	Primary	NOT NULL	Surrogate key
VariantId	INT	Foreign	NOT NULL	Surrogate key
ProblemId	INT	Foreign	NOT NULL	Surrogate key
Attributes	JSON	No	NULL	
InsertedAnswer	TEXT	No	NULL	
CorrectAnswer	JSON	No	NOT NULL	
Result	BOOLEAN	No	NOT NULL	Default false

Таблица 6 — TopicStatistics

Column Name	Type	Key	NULL Status	Remarks
StatID	INT	Primary	NOT NULL	Surrogate key
UserLogins	VARCHAR(50)	Foreign	NOT NULL	Foreign key
TopicName	NVARCHAR(50)	Foreign	NOT NULL	Foreign key
SolvedProblems	INT	No	NOT NULL	Default 0
SuccessRate	FLOAT	No	NOT NULL	От 0 до 1

Объяснение связей таблиц было дано в предыдущем разделе в пояснениях к ER модели.

3. Нормализация реляционной модели

Процесс проектирования реляционной модели базы данных включал этап нормализации — последовательного приведения структуры таблиц к нормальным формам для устранения избыточности данных и аномалий обновления. Нормализация проводилась вплоть до нормальной формы Бойса-Кодда (BCNF).

3.1 Приведение к первой нормальной форме (1NF).

1. Требованием первой нормальной формы является атомарность атрибутов и отсутствие повторяющихся групп.
2. Исходный набор атрибутов был проверен на атомарность. Составные атрибуты, например ФИО, были разделены на атомарные FirstName, LastName, Patronymic. Для разрешения связей «многие-ко-многим» (между сущностями «Вариант» и «Задача» была введена связующая таблица VariantProblem.
3. Полученная модель, состоящая из таблиц Users, Topics, Problem, Variant, VariantProblem и TopicStatistics, удовлетворяет требованиям 1NF.

3.2 Приведение ко второй нормальной форме (2NF).

1. Модель должна находиться в 1NF, и каждый неключевой атрибут должен полностью функционально зависеть от всего первичного ключа.
2. Для каждой таблицы были выявлены функциональные зависимости от первичного ключа.

Таблица 7 — Функциональные зависимости для проверки 2NF.

Таблица	Первичный ключ (PK)	Функциональные зависимости от PK
Users	Logins	Logins → FirstName, LastName, Patronymic, Passwords
Topics	TopicName	TopicName → ProblemAmount, SourceLink
Problem	ProblemId	ProblemId → TopicName, Question, QuestionGenerator, AnswerGenerator
Variant	VariantId	VariantId → UserLogins, CreationTime, Score

VariantProblem	VPID	VPID → VariantId, ProblemId, Attributes, InsertedAnswer, CorrectAnswer, Result
TopicStatistics	StatID	StatID → UserLogins, TopicName, SolvedProblems, SuccessRate

3. Поскольку первичные ключи таблиц являются простыми (одноатрибутными), частичные зависимости невозможны. В таблице VariantProblem неключевые атрибуты функционально полно зависят от всего первичного ключа VPID. Модель удовлетворяет требованиям 2NF.

1. Приведение к третьей нормальной форме (3NF).

1. Модель должна находиться в 2NF, и не должно быть транзитивных зависимостей.
2. В каждой таблице был проведен поиск транзитивных зависимостей. Все неключевые атрибуты зависят непосредственно от первичного ключа. Выделение сущности Topics в отдельную таблицу устранило потенциальную транзитивную зависимость атрибутов темы от TopicName в таблице Problem.

Таблица 8 — Анализ транзитивных зависимостей для проверки 3NF.

Таблица	Потенциальная транзитивная зависимость	Наличие нарушения 3NF
Users	—	—
Topics	—	—
Problem	ProblemId → TopicName → SourceLink	Нет, т.к. TopicName — внешний ключ, а SourceLink находится в другой таблице
Variant	—	—
VariantProblem	—	—

TopicStatistics	—	—
-----------------	---	---

3. Транзитивные зависимости неключевых атрибутов от других неключевых атрибутов в таблицах не выявлены. Модель удовлетворяет требованиям 3NF.

2. Анализ на соответствие нормальной форме Бойса-Кодда (BCNF).

1. Для каждой функциональной зависимости $X \rightarrow Y$, где Y не входит в X , детерминант X должен быть потенциальным ключом.

2. В каждой таблице были проверены все возможные функциональные зависимости. Во всех случаях детерминантом выступает первичный ключ таблицы или альтернативный ключ.

Таблица 9 — Анализ детерминантов для проверки BCNF.

Таблица	Потенциальные ключи (Кандидаты)	Выявленные функциональные зависимости	Соответствие BCNF
Users	Logins	Logins $\rightarrow$ FirstName, LastName, Patronymic, Passwords	Да, детерминант — ключ.
Topics	TopicName	TopicName $\rightarrow$ ProblemAmount, SourceLink	Да, детерминант — ключ.
Problem	ProblemId	ProblemId $\rightarrow$ TopicName, Question, QuestionGenerator, AnswerGenerator	Да, детерминант — ключ.
Variant	VariantId	VariantId $\rightarrow$ UserLogins, CreationTime, Score	Да, детерминант — ключ.
VariantProblem	VPID, (VariantId, ProblemId)	VPID $\rightarrow$ VariantId, ProblemId, Attributes, InsertedAnswer, CorrectAnswer, Result (VariantId, ProblemId) $\rightarrow$ VPID,	Да, оба детерминанта — потенциальные ключи.

		Attributes, InsertedAnswer, CorrectAnswer, Result	
Topic Statistics	StatID, (UserLogins, TopicName)	StatID $\rightarrow$ UserLogins, TopicName, SolvedProblems, SuccessRate (UserLogins, TopicName) $\rightarrow$ StatID, SolvedProblems, SuccessRate	Да, оба детерминанта — потенциальные ключи.

3. Во всех таблицах каждый детерминант является потенциальным ключом. Разработанная модель удовлетворяет требованиям нормальной формы Бойса-Кодда (BCNF).

В результате последовательной нормализации была получена модель, свободная от аномалий вставки, обновления и удаления и приведенная к нормальной форме Бойса-Кодда (BCNF). Данная структура обеспечивает целостность, минимальную избыточность и эффективность хранения данных.

Введение и объяснение необходимых триггеров будет дано в следующем разделе.

С учетом этого, разработанная реляционная модель полностью готова для физической реализации в СУБД и интеграции с серверной логикой приложения.

4. Реализация базы данных

3.1. Физическая модель в SQL

Физическая реализация спроектированной реляционной модели базы данных была выполнена с использованием системы управления базами данных MySQL. Этот выбор обусловлен широкой распространенностью, надежностью и стабильностью данной СУБД, что делает ее подходящим решением для учебных проектов. Процесс реализации заключался не только в прямом переводе сущностей и атрибутов в SQL-структуры, но и в добавлении механизмов, обеспечивающих автоматическое поддержание целостности и консистентности данных непосредственно на уровне сервера базы данных. Эти механизмы реализованы в виде набора автоматически выполняемых триггеров.

Одной из ключевых задач была автоматизация поддержания актуальности счетчика задач в каждой теме. Для этого были созданы два взаимодополняющих триггера. Первый триггер, срабатывающий перед операцией вставки новой записи в таблицу задач, выполняет проверку на существование указанной для задачи темы. Если тема отсутствует в таблице тем, триггер самостоятельно инициирует создание новой записи темы с инициализацией счетчика задач значением один. В случае же существования темы действие триггера ограничивается увеличением значения счетчика на единицу. Второй триггер обеспечивает симметричность логики, активируясь перед удалением задачи и уменьшая счетчик задач в соответствующей теме. Такое решение позволяет гарантировать, что значение счетчика всегда точно отражает текущее состояние базы данных, независимо от того, через какой интерфейс вносятся изменения – через основное клиентское приложение или путем прямого доступа к базе данных. Это критически важно для корректного отображения статистики и исключения рутинных ошибок, связанных с ручным обновлением подобных агрегированных полей.

Помимо ведения счетчиков, важной задачей является защита смысловой целостности данных. Ярким примером служит поле, фиксирующее момент создания тестового варианта. Это значение должно быть установлено один раз в момент генерации варианта и оставаться неизменным в течение всего его жизненного цикла. Для обеспечения этого правила был разработан триггер, который активируется перед любой попыткой обновления записи варианта. Его логика заключается в сравнении старого и нового значения времени создания. В случае обнаружения попытки изменения триггер инициирует специальную ошибку, которая приводит к откату всей транзакции. Таким образом, некорректная операция не может быть выполнена, а данные остаются защищенными от случайной или намеренной порчи на самом фундаментальном уровне.

Далее, для реализации одной из основных функций системы – отслеживания прогресса студента – был создан более сложный триггер, отвечающий за расчет персональной статистики. Этот триггер запускается после того, как фиксируется результат решения студентом очередной задачи. Его работа начинается с определения логина студента и темы решенной задачи на основе идентификаторов, связанных с обновленной записью. Затем осуществляется проверка наличия уже существующей статистики для данной пары «студент-тема». Если запись отсутствует, она создается с начальными значениями. Если же статистика уже ведется, триггер пересчитывает оба ключевых показателя: общее количество решенных задач увеличивается на единицу, а показатель успешности (SuccessRate) обновляется по алгоритму последовательного пересчета среднего балла, который не требует хранения истории всех попыток и вычисляется по формуле “новый рейтинг = (старый рейтинг * количество решенных задач + результат текущей задачи) / (количество решенных

задач + 1)”. Это обеспечивает объективность и точность формируемой статистики.

Наконец, для автоматического определения итоговой оценки за тестовый вариант был реализован триггер, который также срабатывает после обновления результата по каждой задаче в варианте. Его алгоритм включает подсчет общего количества задач в варианте и количества правильно решенных из них. После этого итоговая оценка вычисляется как простая доля правильных ответов от их общего числа и сохраняется в соответствующем поле варианта. Это избавляет от необходимости выполнять данный расчет в клиентском приложении каждый раз при запросе информации о варианте, перенося вычислительную нагрузку на сервер БД и гарантируя, что оценка всегда актуальна.

Таким образом, реализация на уровне SQL позволила закодировать важнейшие бизнес-правила системы непосредственно в схеме данных. Это обеспечивает высочайшую степень надежности их выполнения, поскольку данные правила действуют всегда, при любом способе взаимодействия с базой. Использование триггеров для автоматизации рутинных операций подсчета и контроля целостности данных создает основу для работы клиентских приложений, сводя к минимуму риск появления несогласованностей в данных. Полные скрипты, использованные для создания структуры базы данных и всех описанных триггеров, представлены в приложении А.

3.2. Административное приложение управления базой данных

Административное веб-приложение реализовано для управления базой данных системы тестирования и предназначено для пользователей с правами администратора или преподавателя. Оно предоставляет интерфейс для выполнения ключевых административных функций, которые можно сгруппировать по следующим направлениям:

1. Управления тестовыми задачами:
 - а. просмотр перечня всех задач;
 - б. добавление новых задач в систему;
 - с. удаление существующих задач.
2. Управление темами задач:
 - а. просмотр списка тем;
 - б. создание новых тематических разделов;
 - с. удаление тем вместе со всеми связанными с ними задачами.
3. Работа с пользователями:
 - а. просмотр списка зарегистрированных студентов

Реализация приложения выполнена на языке Python с использованием библиотеки `pymysql` для взаимодействия с СУБД MySQL. На текущем этапе разработки взаимодействие с приложением осуществляется через консольный интерфейс. Исходный код приложения приведен в Приложении Б.

3.3. Клиентское приложение для студентов

Клиентское приложение предназначено для пользования базой данных и для решения задач. Неавторизованные пользователи могут зарегистрироваться в системе и авторизоваться. Авторизованные пользователи могут посмотреть статистику или сгенерировать и решить

вариант. При генерации варианта, студенту будет показан список тем с количеством доступных задач по каждой теме. Студент выбирает тему и количество задач из этой темы, которые добавятся в его вариант (рисунок 3).

```
=== Список тем ===
1. Детерминированные конечные автоматы - Задач: 1
2. Регулярные языки - Задач: 1

Выберите тему (номер) или 0 для завершения: 1
Количество задач для 'Детерминированные конечные автоматы' (макс. 1): 1

Выберите тему (номер) или 0 для завершения: 2
Количество задач для 'Регулярные языки' (макс. 1): 1

Выберите тему (номер) или 0 для завершения: 0
```

Рисунок 3 — Выбор задач для варианта.

Из каждой выбранной темы находятся случайные задачи в выбранном количестве. После этого для каждой из этих задач, если у нее предусмотрены атрибуты, они генерируются через QuestionGenerator, после чего эти атрибуты отправляются в AnswerGenerator, чтобы функция CorrectAnswer посчитала правильный ответ от этих параметров. Если вдруг от этих параметров нельзя получить правильный ответ, ошибка возникнет на этапе генерации варианта, что предотвратит попадание некорректной задачи студенту (листинг 3.3.1).

Листинг 3.3.1 – Проверка корректности генерации ответа

```
try:
    correct_answer = subprocess.check_output(
        ['python3', answer_generator, "CorrectAnswer",
         json.dumps(attributes_json)],
        stderr=subprocess.DEVNULL # Перенаправляем stderr, чтобы
        избежать отладочных сообщений
    ).decode('utf-8').strip()
except subprocess.CalledProcessError as e:
    print(f"Ошибка при выполнении генератора ответа:
          {e.output.decode('utf-8')}")
```

```
return False
```

Когда каждая задача полностью сгенерирована, эти задачи и объединяющий их вариант, а также время создания варианта (оно же считается временем начала его решения) отправляются в базу данных одной транзакцией. После этого вариант по одной задаче выдается студенту. Студент вводит однострочный ответ, ответ отправляется в AnswerGenerator, где функция CheckAnswer проверяет его на правильность. В базу данных добавляется ответ студента и результат от AnswerGenerator (true или false), после чего студенту выдается следующая задача (рисунок 4)

```
Вопрос: Построить минимальный ДКА, принимающий числа в степени счисления у (соответственно алфавитом входных символов являются цифры от 0 до у), кратных х.
Формат ответа однострочный, то есть:
{состояния}:{начальное состояние}:{конечные состояния}:{переходы}
Например:
q0,q1,q2:q0:q2,q1:({q0,0}->q1;({q0,1}->q0;({q1,0}->q2;({q1,1}->q1;({q2,0}->q2;({q2,1}->q2
Параметры: {'x': 2, 'y': 3}
Ваш ответ: a,b:a:b:(a,0)->b

Вопрос: Пусть язык L определяется как  $L = \{w \mid |w|_a \neq 2 * |w|_b \text{ и } w \in \{a, b, c\}^*\}$ 
Является ли язык регулярным? Является ли язык контекстно свободным? (Ответ вводится как да или нет через запятую)
Ваш ответ: Нет, да
```

Рисунок 4 — Решение варианта

По окончании решения студентом варианта, его результаты, а также правильный ответ (если студент) решил задачу неправильно.

```
=== Результаты варианта #5 ===
Время выполнения: 0.2 минут
```

```
1. Построить минимальный ДКА, принимающий числа в степени счисления у (соответственно алфавитом входных символов являются цифры от 0 до у), кратных х.
Формат ответа однострочный, то есть:
{состояния}:{начальное состояние}:{конечные состояния}:{переходы}
Например:
q0,q1,q2:q0:q2,q1:({q0,0}->q1;({q0,1}->q0;({q1,0}->q2;({q1,1}->q1;({q2,0}->q2;({q2,1}->q2
С параметрами: {'x': 5, 'y': 2}
Ваш ответ: qq
Результат: Неверно
Правильный ответ: {"states": ["q0", "q1", "q2", "q3", "q4"], "alphabet": ["0", "1"], "start_state": "q0", "transitions": {"q0,0": "q0", "q0,1": "q1", "q1,0": "q2", "q1,1": "q3", "q2,0": "q4", "q2,1": "q4"}, "accept_states": ["q0"]}]

2. Пусть язык L определяется как  $L = \{w \mid |w|_a \neq 2 * |w|_b \text{ и } w \in \{a, b, c\}^*\}$ 
Является ли язык регулярным? Является ли язык контекстно свободным? (Ответ вводится как да или нет через запятую)
Ваш ответ: Нет, да
Результат: Верно
Ваша оценка: 50.00%
```

Рисунок 5 — Результаты варианта

Полный код приложения, написанный на python, с использованием библиотеки pymysql для подключения к базе данных указан в приложении

В. Интерфейса у приложения на данный момент нет, взаимодействие происходит через консоль.

4. Тестирование системы

Для проверки работы системы была добавлена тестовая параметризуемая задача с генерируемыми параметрами. Ниже приведен ее шаблон в формате json.

```
{
  "topic_name": "Детерменированные конечные автоматы",
  "question": "Построить минимальный ДКА, принимающий числа в степени счисления у (соответственно алфавитом входных символов являются цифры от 0 до у), кратных х.\nФормат ответа однострочный, то есть:\n{состояния}:{начальное состояние}:{конечные состояния}:{переходы}\nНапример:\nq0,q1,q2:q0:q2,q1:(q0,0)->q1;(q0,1)->q0;(q1,0)->q2;(q1,1)->q1;(q2,0)->q2;(q2,1)->q2",
  "question_generator": "Problems/2/Program2Question.py",
  "answer_generator": "Problems/2/Program2AnswerCheck.py"
}
```

Для такой задачи генератор должен выбрать 2 небольших целых числа.

```
from random import randint
import json

def main():
    bad = True
    while bad:
        x = randint(2, 10)
        y = randint(2, 10)
        if 5 <= x * y <= 10:
            bad = False
        return {"x": x, "y": y}

if __name__ == "__main__":
    result = main()
    print(json.dumps(result))
```

Program2AnswerCheck должен быть немного сложнее. Ему нужны 2 функции, одна из которых генерирует правильный ответ, другая проверяет введенный ответ на правильность. Они регулируются следующим образом:

```
if __name__ == "__main__":
    if len(sys.argv) < 2:
        print(json.dumps({"error": "Usage: python script.py <function_name> [args...]"}))
        sys.exit(1)

    function_name = sys.argv[1]
```

```

if function_name == "CorrectAnswer":
    args_json = sys.argv[2]
    args = json.loads(args_json)
    x = args['x']
    y = args['y']
    result = CorrectAnswer(x, y)
    print(json.dumps(result))

elif function_name == "CheckAnswer":
    args_json = sys.argv[2]
    args = json.loads(args_json)
    x = args['x']
    y = args['y']
    inserted_dfa_json = sys.argv[3]
    result = CheckAnswer(x, y, inserted_dfa_json)
    print(json.dumps({"result": result}))

else:
    print(json.dumps({"error": f"Unknown function: {function_name}"}))

```

Полный код программы Program2AnswerCheck на всякий случай указан в приложении Г.

5. Заключение

В ходе выполнения данной курсовой работы была разработана система тестирования с динамической генерацией задач, включающая проектирование базы данных, реализацию серверного и клиентского приложений, а также добавлении в систему задач.

Разработанная система соответствует поставленным требованиям и решает задачи автоматизации проверки знаний студентов. Использование динамической генерации задач и разделения уровней доступа (для студентов и преподавателей) обеспечивает гибкость, масштабируемость и надёжность системы. В дальнейшем возможно расширение функциональности за счёт добавления веб-интерфейса или мобильного приложения, что сделает систему ещё более удобной для пользователей.

Список литературы

1. Diagrams.net [Электронный ресурс] // Онлайн-инструмент для создания диаграмм. — Режим доступа: <https://app.diagrams.net/> (дата обращения: 13.09.2025).
2. QuickDBD [Электронный ресурс] // Онлайн-инструмент для проектирования реляционных баз данных. — Режим доступа: <https://www.quickdatabasediagrams.com/> (дата обращения: 13.09.2025).
3. Visual Studio Code [Электронный ресурс] // Редактор кода от Microsoft. — Режим доступа: <https://code.visualstudio.com/> (дата обращения: 28.03.2025).
4. PyMySQL [Электронный ресурс] // Библиотека для работы с MySQL в Python. — Режим доступа: <https://pypi.org/project/PyMySQL/> (дата обращения: 30.03.2025).
5. Репозиторий задач [Электронный ресурс] // GitHub-репозиторий с интересными задачами по Теории Формальных Языков. — Режим доступа: [<https://github.com/TonitaN/FormalLanguageTheory>] (дата обращения: 12.09.2025).

Приложение А

--Создание базы данных

```
DROP DATABASE IF EXISTS test_db;
```

```
CREATE DATABASE test_db;
```

```
USE test_db;
```

--Таблица пользователей

```
DROP TABLE IF EXISTS Users;
```

```
CREATE TABLE Users (  
Logins VARCHAR(50) PRIMARY KEY,  
FirstName NVARCHAR(50) NOT NULL,  
LastName NVARCHAR(50) NOT NULL,  
Patronymic NVARCHAR(50) NULL,  
Passwords VARCHAR(255) NOT NULL  
);
```

--Таблица тем задач

```
DROP TABLE IF EXISTS Topics;
```

```
CREATE TABLE Topics (  
TopicName NVARCHAR(50) PRIMARY KEY,  
ProblemAmount INT NOT NULL DEFAULT(0),  
SourceLink VARCHAR(2048) NULL  
);
```

--Таблица задачи, с каскадным ее удалением, в случае если темы этой задачи больше
--нет

```
DROP TABLE IF EXISTS Problem;
```

```
CREATE TABLE Problem (  
ProblemId INT AUTO_INCREMENT PRIMARY KEY,  
TopicName NVARCHAR(50),  
Question TEXT NOT NULL,  
QuestionGenerator VARCHAR(512) NULL,  
AnswerGenerator VARCHAR(512) NOT NULL,  
FOREIGN KEY (TopicName) REFERENCES Topics(TopicName) ON DELETE  
CASCADE  
);
```

--Таблица варианта теста, с аналогичным каскадным удалением

```
DROP TABLE IF EXISTS Variant;
```

```
CREATE TABLE Variant (  
VariantId INT AUTO_INCREMENT PRIMARY KEY,  
UserLogins VARCHAR(50),  
CreationTime DATETIME NULL,  
Score FLOAT DEFAULT NULL,  
FOREIGN KEY (UserLogins) REFERENCES Users(Logins) ON DELETE CASCADE  
);
```

--Таблица задачи в варианте. Каскадное удаление происходит по варианту, а не по

--задаче, потому что удаление задачи во время прохождения теста
--студентом могло бы привести к потере данных или ошибкам.

```
DROP TABLE IF EXISTS VariantProblem;  
CREATE TABLE VariantProblem (  
  VPID INT AUTO_INCREMENT PRIMARY KEY,  
  VariantId INT,  
  ProblemId INT,  
  Attributes JSON NULL,  
  InsertedAnswer TEXT NULL,  
  CorrectAnswer JSON NOT NULL,  
  Result BOOLEAN NOT NULL DEFAULT FALSE,  
  FOREIGN KEY (ProblemId) REFERENCES Problem(ProblemId),  
  FOREIGN KEY (VariantId) REFERENCES Variant(VariantId) ON DELETE CASCADE  
);
```

--Таблица статистики студента по теме. Если преподаватель или владелец сервера
--счел тему неактуальной, это не значит, что студенту стоит забыть о своих
--достижениях в этой теме.

```
DROP TABLE IF EXISTS TopicStatistics;  
CREATE TABLE TopicStatistics(  
  StatID INT AUTO_INCREMENT PRIMARY KEY,  
  UserLogins VARCHAR(50),  
  TopicName NVARCHAR(50),  
  SolvedProblems INT NOT NULL DEFAULT 0,  
  SuccessRate FLOAT NOT NULL DEFAULT 0,  
  FOREIGN KEY (UserLogins) REFERENCES Users(Logins) ON DELETE CASCADE,  
  FOREIGN KEY (TopicName) REFERENCES Topics(TopicName)  
);
```

--Триггер на вставку нового шаблона задачи, сохраняя их количество в темах
DROP TRIGGER IF EXISTS handleProblemInsert;

```
DELIMITER //  
CREATE TRIGGER handleProblemInsert BEFORE INSERT ON Problem  
FOR EACH ROW  
BEGIN  
  -- 1. Проверяем, существует ли тема  
  IF (SELECT COUNT(*) FROM Topics WHERE TopicName = NEW.TopicName) = 0  
  THEN  
    -- Если темы нет, создаём её с ProblemAmount = 1  
    INSERT INTO Topics (TopicName, ProblemAmount, SourceLink)  
    VALUES (NEW.TopicName, 1, NULL);  
  ELSE  
    -- 2. Если тема существует, увеличиваем счётчик  
    UPDATE Topics  
    SET ProblemAmount = ProblemAmount + 1  
    WHERE TopicName = NEW.TopicName;  
  END IF;  
END //  
DELIMITER ;
```

--Удаление шаблона задачи так же влияет на запись их количества в темах

```

DROP TRIGGER IF EXISTS deletedProblem;
DELIMITER //
CREATE TRIGGER deletedProblem BEFORE DELETE ON Problem
FOR EACH ROW
BEGIN
UPDATE Topics
SET ProblemAmount = ProblemAmount - 1
WHERE TopicName = OLD.TopicName;
END //
DELIMITER ;

--Триггер о неизменности времени создания варианта
DELIMITER //
CREATE TRIGGER check_creation_time_update
BEFORE UPDATE ON Variant
FOR EACH ROW
BEGIN
-- Проверяем, что CreationTime не изменяется
IF NEW.CreationTime != OLD.CreationTime AND NEW.CreationTime IS NOT NULL
THEN
SIGNAL SQLSTATE '45000'
SET MESSAGE_TEXT = 'CreationTime может быть установлено только один раз.';
END IF;
END//
DELIMITER ;

--Триггер на изменение статистики пользователя - когда он решил задачу
DROP TRIGGER IF EXISTS updateTopicStatistics;
DELIMITER //
CREATE TRIGGER updateTopicStatistics AFTER UPDATE ON VariantProblem
FOR EACH ROW
BEGIN
DECLARE user_login VARCHAR(50);
DECLARE topic_name VARCHAR(50);
DECLARE solved_problems INT;
DECLARE success_rate FLOAT;

-- Получаем UserLogins из Variant и TopicName из Problem
SELECT UserLogins INTO user_login FROM Variant WHERE VariantId = NEW.VariantId;
SELECT TopicName INTO topic_name FROM Problem WHERE ProblemId =
NEW.ProblemId;

-- Проверяем, существует ли статистика для пользователя и темы
SELECT COUNT(*) INTO @count FROM TopicStatistics
WHERE UserLogins = user_login AND TopicName = topic_name;

IF @count = 0 THEN
-- Создаем новую запись в статистике
INSERT INTO TopicStatistics (UserLogins, TopicName, SolvedProblems, SuccessRate)
VALUES (user_login, topic_name, 1, IF(NEW.Result, 1, 0));
ELSE

```

```

-- Обновляем существующую запись в статистике
UPDATE TopicStatistics
SET
SuccessRate = (SuccessRate * SolvedProblems + IF(NEW.Result, 1, 0)) / (SolvedProblems
+ 1),
SolvedProblems = SolvedProblems + 1
WHERE UserLogins = user_login AND TopicName = topic_name;
END IF;
END //
DELIMITER ;

--В варианте всегда подсчитывается промежуточная оценка студента
DROP TRIGGER IF EXISTS updateVariant;
DELIMITER //
CREATE TRIGGER updateVariant AFTER UPDATE ON VariantProblem
FOR EACH ROW
BEGIN
DECLARE total_problems INT;
DECLARE correct_problems INT;

-- Получаем общее количество задач и количество правильных ответов
SELECT COUNT(*) INTO total_problems FROM VariantProblem WHERE VariantId =
NEW.VariantId;
SELECT COUNT(*) INTO correct_problems FROM VariantProblem
WHERE VariantId = NEW.VariantId AND Result = TRUE;

-- Обновляем оценку в таблице Variant
UPDATE Variant
SET Score = correct_problems / total_problems
WHERE VariantId = NEW.VariantId;
END //
DELIMITER ;

```

Приложение Б

```
import pymysql
import json

#Подключение к базе данных
class DatabaseManager:
    def __init__(self):
        self.db = pymysql.connect(
            host='localhost',
            user='root',
            password='123',
            database='test_db'
        )

#Функция добавления шаблона задачи работает через json файл, потому что проще
#заранее написать условие задачи в файле, чем писать в консоли все это в одну
#строчку
    def add_Problem(self, file_path):
        with open(file_path, 'r') as file:
            task_data = json.load(file)

        cursor = self.db.cursor()
        try:
            cursor.execute(
                "INSERT INTO Problem (TopicName, Question, AnswerGenerator, "
                "QuestionGenerator) "
                "VALUES (%s, %s, %s, %s)",
                (
                    task_data['topic_name'],
                    task_data['question'],
                    task_data['answer_generator'], task_data.get('question_generator', "")
                )
            )
            self.db.commit()
            print("Задача успешно добавлена из файла.")
        except Exception as e:
            self.db.rollback()
            print(f"Ошибка: {e}")
        finally:
            cursor.close()

#Функция для удаления устаревшей задачи
    def delete_Problem(self, problem_id):
        cursor = self.db.cursor()
        try:
            cursor.execute(
                "DELETE FROM Problem WHERE ProblemId=%s",
                (problem_id,)
            )
```

```

        self.db.commit()
        print("Задача успешно удалена.")
    except Exception as e:
        self.db.rollback()
        print(f"Ошибка: {e}")
    finally:
        cursor.close()

```

#Функция добавления темы задач

```

def add_Topic(self, topic_name, SourceLink=None):
    cursor = self.db.cursor()
    try:
        cursor.execute(
            "INSERT INTO Topics (TopicName, SourceLink) VALUES (%s, %s)",
            (topic_name, SourceLink)
        )
        self.db.commit()
        print("Тема успешно добавлена.")
    except Exception as e:
        self.db.rollback()
        print(f"Ошибка: {e}")
    finally:
        cursor.close()

```

#Функция удаления темы со всеми ее задачами

```

def delete_Topic(self, topic_name):
    cursor = self.db.cursor()
    try:
        cursor.execute(
            "DELETE FROM Topics WHERE TopicName=%s",
            (topic_name,)
        )
        self.db.commit()
        print("Тема успешно удалена.")
    except Exception as e:
        self.db.rollback()
        print(f"Ошибка: {e}")
    finally:
        cursor.close()

```

#Функция для просмотра тем задач, из базы данных, будет полезна чтобы определить
#к чему отнести свою задачу.

```

def show_all_topics(self):
    cursor = self.db.cursor()
    try:
        cursor.execute("SELECT * FROM Topics")
        topics = cursor.fetchall()
        if not topics:
            print("Темы не найдены.")
        return

```

```

        print("Список тем:\n")
        for topic in topics:
            print(f"Название темы: {topic[0]}\nКоличество задач: {topic[1]}")
            if topic[2]: # Проверяем, что ссылка не пустая и не None
                print(f"Ссылка на лекцию: {topic[2]}\n")
            print()
    except Exception as e:
        print(f"Ошибка: {e}")
    finally:
        cursor.close()

#Функция показывает все задачи указанной темы, либо вообще все задачи, если тема
#не указана
def show_all_problems(self, topic_name=None):
    cursor = self.db.cursor()
    try:
        if topic_name:
            cursor.execute(
                "SELECT * FROM Problem WHERE TopicName = %s",
                (topic_name,)
            )
        else:
            cursor.execute("SELECT * FROM Problem")
        problems = cursor.fetchall()
        if not problems:
            print("Задачи не найдены.")
            return
        print("Список задач:\n")
        for problem in problems:
            print(f"ID: {problem[0]}\nВопрос: {problem[2]}\n")
    except Exception as e:
        print(f"Ошибка: {e}")
    finally:
        cursor.close()

#Функция для просмотра всех зарегистрированных в системе студентов
def show_users(self):
    cursor = self.db.cursor()
    try:
        cursor.execute("SELECT * FROM Users")
        users = cursor.fetchall()
        print("Список пользователей:\n")
        for user in users:
            print(f"Login: {user[0]}")
            if not user[3]:
                print(f"Имя: {user[2]} {user[1]}\n")
            else:
                print(f"Имя: {user[2]} {user[1]} {user[3]}\n")
    except Exception as e:
        print(f"Ошибка: {e}")
    finally:

```

```

        cursor.close()
#Выход из приложения = закрытие соединения с базой данных
    def close(self):
        self.db.close()
#Main отвечает за взаимодействие человека с приложением
def main():
    db_manager = DatabaseManager()
    while True:
        action = input("Выберите действие: 1 - Добавить задачу, 2 - Добавить
        тему, 3 - Удалить задачу, 4 - Удалить тему, 5 - Показать все темы, 6 -
        Показать задачи, 7 - Показать всех пользователей, 0 - Выйти: ")
        if action == '1':
            filepath = input("Введите ссылку на шаблон задачи: ")
            db_manager.add_Problem(filepath)
        elif action == '2':
            topic_name = input("Введите название темы: ")
            SourceLink = input("Введите ссылку на тему: ")
            db_manager.add_Topic(topic_name, SourceLink)
        elif action == '3':
            problem_id = int(input("Введите номер задачи: "))
            db_manager.delete_Problem(problem_id)
        elif action == '4':
            topic_name = input("Введите название темы: ")
            db_manager.delete_Topic(topic_name)
        elif action == '5':
            db_manager.show_all_topics()
        elif action == '6':
            topic_name = input("Введите название темы: ")
            db_manager.show_all_problems(topic_name)
        elif action == '7':
            db_manager.show_users()
        elif action == '0':
            break
    db_manager.close()
if __name__ == "__main__":
    main()

```

Приложение В

```
import json
import pymysql
import subprocess
import random
import datetime
import os

#Подключение к той же базе данных
def get_db_connection():
    return pymysql.connect(
        host='localhost',
        user='root',
        password='123',
        database='test_db'
    )

#Функция добавления пользователя происходит через несколько строк, а не json,
# потому что на данный момент студент якобы не может создавать типовые файлы #для
этого. В будущем, в потенциальном web приложении будет использоваться json, #
создаваемый на сайте.
def add_user():
    print("\n=== Регистрация нового пользователя ===")
    first_name = input("Имя: ")
    last_name = input("Фамилия: ")
    patronimic_input = input("Отчество (опционально): ")
    patronimic = patronimic_input if patronimic_input else None
    login = input("Логин: ")
    password = input("Пароль: ")
    db = get_db_connection()
    cursor = db.cursor()
    try:
        cursor.execute(
            "INSERT INTO Users (FirstName, LastName, Logins, Passwords,
            Patronymic) VALUES (%s, %s, %s, %s, %s)",
            (first_name, last_name, login, password, patronimic)
        )
        db.commit()
        print("Пользователь успешно добавлен.")
        return True
    except Exception as e:
        db.rollback()
        print(f"Ошибка: {e}")
        return False
    finally:
        cursor.close()
        db.close()
```

#Функция авторизации возвращает все данные пользователя, потому что некоторые
#из них используются в функциях для авторизованных студентов

```
def authenticate_user():
    print("\n=== Авторизация ===")
    login = input("Логин: ")
    password = input("Пароль: ")

    db = get_db_connection()
    cursor = db.cursor()
    try:
        cursor.execute(
            "SELECT COUNT(*) FROM Users WHERE Logins = %s",
            (login,)
        )
        exists = cursor.fetchone()[0]
        if exists == 0:
            print("Неверный логин.")
            return None
        else:
            cursor.execute(
                "SELECT * FROM Users WHERE Logins = %s AND Passwords = %s",
                (login, password)
            )
            user = cursor.fetchone()
            if user:
                print("Авторизация успешна!")
                return user
            else:
                print("Неверный пароль.")
                return None
    except Exception as e:
        print(f"Ошибка: {e}")
        return None
    finally:
        cursor.close()
        db.close()

#Функция get_topics сначала выводит темы, а потом возвращает их для других функций
def get_topics():
    db = get_db_connection()
    cursor = db.cursor()
    cursor.execute("SELECT * FROM Topics")
    topics = cursor.fetchall()
    cursor.close()
    db.close()
    print("\n=== Список тем ===")
    for i, topic in enumerate(topics, 1):
        print(f"{i}. {topic[0]} - Задач: {topic[1]}")
        if topic[2]:
            print(f"Ссылка на лекцию: {topic[2]}")
    return topics
```

```

#Функция для генерации задачи по шаблоны вызывается при создании варианта
def add_variant_problem (cursor, db, variant_id, problem_id):
    try:
        # Получаем данные задачи по ProblemId
        cursor.execute(
            "SELECT QuestionGenerator, AnswerGenerator FROM Problem WHERE
            ProblemId = %s",
            (problem_id,)
        )
        problem = cursor.fetchone()
        if not problem:
            print(f"Задача с ProblemId {problem_id} не найдена.")
            return False

        question_generator, answer_generator = problem

        # Проверяем, что пути к генераторам существуют, если они не пустые
        if question_generator:
            if not os.path.exists(question_generator):
                return False
        else:
            attributes_json = {}

        if answer_generator and not os.path.exists(answer_generator):
            return False

        # Если QuestionGenerator отсутствует, пропускаем генерацию атрибутов
        if question_generator:
            # Запускаем генератор вопроса для получения атрибутов
            try:
                attributes = subprocess.check_output(
                    ['python3', question_generator],
                    stderr=subprocess.DEVNULL
                ).decode('utf-8').strip()

                # Преобразуем атрибуты в JSON
                try:
                    attributes_json = json.loads(attributes)
                except json.JSONDecodeError as e:
                    print(f"Ошибка декодирования JSON: {e}")
                    return False
            except subprocess.CalledProcessError as e:
                print(f"Ошибка при выполнении генератора вопроса:
                {e.output.decode('utf-8')}")
                return False
        else:
            attributes_json = {}

        # Запускаем генератор ответа с атрибутами
        try:

```

```

        correct_answer = subprocess.check_output(
            ['python3', answer_generator, "CorrectAnswer",
            json.dumps(attributes_json)],
            stderr=subprocess.DEVNULL # Перенаправляем stderr, чтобы
            избежать отладочных сообщений
        ).decode('utf-8').strip()
    except subprocess.CalledProcessError as e:
        print(f"Ошибка при выполнении генератора ответа:
        {e.output.decode('utf-8')}")
        return False

# Вставляем запись в VariantProblem
try:
    cursor.execute(
        "INSERT INTO VariantProblem (VariantId, ProblemId, Attributes,
        CorrectAnswer) VALUES (%s, %s, %s, %s)",
        (variant_id, problem_id, json.dumps(attributes_json), correct_answer)
    )
    db.commit()
    return True
except Exception as e:
    print(f"Ошибка при добавлении записи в базу данных: {e}")
    db.rollback()
    return False

except Exception as e:
    print(f"Неожиданная ошибка: {e}")
    db.rollback()
    return False

```

#Функция создания варианта

```

def create_variant(user_login):
    print("\n=== Создание варианта ===")
    #Вот здесь используются темы, которые мы возвращали в другой функции
    topics = get_topics()
    selected_topics = []
    #Студент будет выбирать задачи из тем, которые увидит. Число выбранных
    #задач будет не больше, чем количество существующих в базе данных задач по
    #этой теме
    while True:
        try:
            topic_choice = input("\nВыберите тему (номер) или 0 для
            завершения: ")
            if topic_choice == '0':
                break

            topic_index = int(topic_choice) - 1
            if 0 <= topic_index < len(topics):
                topic_name = topics[topic_index][0]
                max_problems = topics[topic_index][1]

```

```

        problem_count = int(input(f'Количество задач для
        '{topic_name}' (макс. {max_problems}): '))
        if problem_count > max_problems:
            print(f'Нельзя выбрать больше {max_problems} задач')
            continue
        if problem_count <= 0:
            print("Количество задач должно быть положительным")
            continue

        selected_topics.append({
            'topic_name': topic_name,
            'problem_count': problem_count
        })
    else:
        print("Неверный номер темы")
except ValueError:
    print("Введите число")

if not selected_topics:
    print("Не выбрано ни одной темы")
    return None

db = get_db_connection()
cursor = db.cursor()
try:
    # Создаем новый вариант
    cursor.execute(
        "INSERT INTO Variant (UserLogins) VALUES (%s)",
        (user_login,)
    )
    variant_id = cursor.lastrowid

    # Для каждого типа задач выбираем случайные задачи и добавляем их в
    вариант
    for topic in selected_topics:
        topic_name = topic['topic_name']
        problem_count = topic['problem_count']

        # Получаем список задач данного типа
        cursor.execute(
            "SELECT ProblemId FROM Problem WHERE TopicName = %s",
            (topic_name,)
        )
        problems = cursor.fetchall()

        if len(problems) < problem_count:
            continue

        # Выбираем случайные задачи
        selected_problems = random.sample(problems, problem_count)

```

```

        # Добавляем задачи в вариант
        for problem in selected_problems:
            problem_id = problem[0]
            if not add_variant_problem(cursor, db, variant_id,
                                      problem_id):
                continue

    # Добавляем время создания варианта
    cursor.execute(
        "UPDATE Variant SET CreationTime = NOW() WHERE VariantId = %s",
        (variant_id,)
    )
    db.commit()
    return variant_id
except Exception as e:
    db.rollback()
    print(f"Ошибка: {e}")
    return None
finally:
    cursor.close()
    db.close()

```

#После этого студент рашает свой вариант

```

def solve_variant_problems(variant_id):
    db = get_db_connection()
    cursor = db.cursor()
    try:
        cursor.execute(
            "SELECT VPID, ProblemId, Attributes FROM VariantProblem WHERE
            VariantId = %s",
            (variant_id,)
        )
        problems = cursor.fetchall()
        print(f"\n=== Решение варианта #{variant_id} ===")
        print("Для каждой задачи введите ответ и нажмите Enter")
        print()
        for vp_id, problem_id, attributes_json in problems:
            cursor.execute(
                "SELECT Question FROM Problem WHERE ProblemId = %s",
                (problem_id,)
            )
            question = cursor.fetchone()[0]
            attributes = json.loads(attributes_json)
            print(f"\nВопрос: {question}")
            if attributes:
                print("Параметры:", attributes)
            answer = input("Ваш ответ: ").strip()
            cursor.execute(
                "SELECT AnswerGenerator FROM Problem WHERE ProblemId =
                %s",
                (problem_id,)
            )

```

```

    )
    answer_generator_path = cursor.fetchone()[0]
    try:
        result_output = subprocess.check_output(
            ['python3', answer_generator_path, "CheckAnswer",
            attributes_json, answer],
            stderr=subprocess.DEVNULL
        ).decode('utf-8').strip()
        result_json = json.loads(result_output)
        result = result_json.get("result", False)
    except Exception as e:
        print(f"Ошибка при проверке ответа: {e}")
        result = "error"

    cursor.execute(
        "UPDATE VariantProblem SET InsertedAnswer = %s, Result = %s
        WHERE VPID = %s",
        (answer, result, vp_id)
    )
    db.commit()
end_time = datetime.datetime.now()
cursor.execute(
    "SELECT CreationTime FROM Variant WHERE VariantId = %s",
    (variant_id,)
)
start_time = cursor.fetchone()[0]
time_taken = end_time - start_time
return time_taken
except Exception as e:
    db.rollback()
    print(f"Ошибка: {e}")
finally:
    cursor.close()
    db.close()

#После решения последней задачи, студент может увидеть, что он решил правильно, а
#что нет.
def show_variant_results(variant_id, time_taken):
    print(f"\n=== Результаты варианта #{variant_id} ===")
    if time_taken:
        minutes = time_taken.total_seconds() / 60
        print(f"Время выполнения: {minutes:.1f} минут")
        print()

db = get_db_connection()
cursor = db.cursor()
try:
    cursor.execute(
        """SELECT p.Question, vp.Attributes, vp.InsertedAnswer,
        vp.CorrectAnswer, vp.Result
        FROM VariantProblem vp
        JOIN Problem p ON vp.ProblemId = p.ProblemId

```

```

WHERE vp.VariantId = %s""",
(variant_id,)
)
problems = cursor.fetchall()
for i, (question, attributes, inserted, correct, result) in enumerate(problems, 1):
    print(f"\n{i}. {question}")
    if attributes and attributes != '[]' and attributes != [] and attributes.strip()
    != "" and attributes != "{}":
        print(f"С параметрами: {attributes}")
    print(f"Ваш ответ: {inserted}")
    print(f"Результат: {'Верно' if result else 'Неверно'}")
    # Показываем правильный ответ только если ответ неправильный
    if result == 'False':
        print(f"Правильный ответ: {correct}")
    cursor.execute(
        "SELECT Score FROM Variant WHERE VariantId = %s",
        (variant_id,)
    )
    score = cursor.fetchone()[0]

    if score is not None:
        print(f"Ваша оценка: {score*100:.2f}%")
    else:
        print("Ваша оценка еще не рассчитана")
except Exception as e:
    print(f"Ошибка: {e}")
finally:
    cursor.close()
    db.close()

```

#Весь процесс создания и решения варианта непрерывен, и запускается из одной функции

```

def create_and_solve_variant(user_login):
    variant_id = create_variant(user_login)
    if variant_id:
        time_taken = solve_variant_problems(variant_id)
        show_variant_results(variant_id, time_taken)
        delete_variant(variant_id)
#Функция удаления варианта
def delete_variant(variant_id):
    db = get_db_connection()
    cursor = db.cursor()
    try:
        # Удаляем вариант с указанным id
        cursor.execute(
            "DELETE FROM Variant WHERE VariantId = %s",
            (variant_id,)
        )
    
```

```

        db.commit()
except Exception as e:
    db.rollback()
    print(f"Ошибка: {e}")
finally:
    cursor.close()
    db.close()

```

#Авторизованный пользователь имеет право удалить свой аккаунт

```

def delete_user(login):
    print("\n=== Удаление пользователя ===")

    db = get_db_connection()
    cursor = db.cursor()
    try:
        cursor.execute(
            "DELETE FROM Users WHERE Logins = %s",
            (login,)
        )
        db.commit()

        if cursor.rowcount > 0:
            print(f"Пользователь '{login}' успешно удален.")
        else:
            print(f"Пользователь с логином '{login}' не найден.")
    except Exception as e:
        db.rollback()
        print(f"Ошибка: {e}")
    finally:
        cursor.close()
        db.close()

```

#Функция выдающая статистику по темам красивой таблицей

```

def show_statistics(user_login):
    print("\n=== Ваша статистика по темам ===")
    db = get_db_connection()
    cursor = db.cursor()
    try:
        cursor.execute(
            """SELECT TopicName, SolvedProblems, SuccessRate
            FROM TopicStatistics
            WHERE UserLogins = %s
            ORDER BY TopicName""",
            (user_login,)
        )
        stats = cursor.fetchall()
        if not stats:
            print("Статистика пока отсутствует. Решите несколько вариантов!")

```

```

        return
    print(f'{'Тема':<30} {'Решено задач':<15} {'Успешность':<15} ")
    print("-" * 60)
    total_solved = 0
    total_success = 0.0
    for topic_name, solved_problems, success_rate in stats:
        print(f'{'topic_name':<30} {'solved_problems':<15}
              {'success_rate*100:<15.1f}%")
        total_success = (total_solved * total_success + success_rate *
                        solved_problems)/(total_solved + solved_problems)
        total_solved += solved_problems
    if total_solved > 0:
        print()
        print(f'{'Всего':<30} {'total_solved':<15}
              {'total_success*100:<15.1f}%")
except Exception as e:
    print(f"Ошибка при получении статистики: {e}")
finally:
    cursor.close()
    db.close()

```

#Функция, временно выполняющая роль интерфейса

```

def main():
    current_user = None
    while True:
        print("\nСистема тестирования")
        print()
        if current_user:
            if not current_user[3]:
                print(f'Текущий пользователь: {current_user[2]}
                      {current_user[1]}")
            else:
                print(f'Текущий пользователь: {current_user[2]}
                      {current_user[1]} {current_user[3]}")
            print("1. Создать и решить новый вариант")
            print("2. Посмотреть свою статистику")
            print("3. Показать доступные темы")
            print("4. Выйти из аккаунта")
            print("5. Удалить пользователя")
            print("0. Выход из системы")
        else:
            print("1. Регистрация")
            print("2. Авторизация")
            print("3. Показать темы")
            print("0. Выход")
        choice = input("Выберите действие: ")
        if current_user:
            if choice == '1':
                create_and_solve_variant(current_user[0])
            elif choice == '2':
                show_statistics(current_user[0])

```

```

elif choice == '3':
    get_topics()
elif choice == '4':
    current_user = None
    print("Вы вышли из аккаунта")
elif choice == '5':
    delete_user(current_user[0])
elif choice == '0':
    print("До свидания!")
    break
else:
    print("Неверный выбор")
else:
    if choice == '1':
        add_user()
    elif choice == '2':
        current_user = authenticate_user()
    elif choice == '3':
        get_topics()
    elif choice == '0':
        print("До свидания!")
        break
    else:
        print("Неверный выбор")

if __name__ == '__main__':
    main()

```

Приложение Г

```
from itertools import permutations
import sys
import json
```

#Более того, задача требует написания автомата и должна уметь работать с ними

```
class DFA:
```

```
    def __init__(self, states, alphabet, transitions, start_state, accept_states):
        self.states = set(states)
        self.alphabet = set(alphabet)
        self.transitions = transitions
        self.start_state = start_state
        self.accept_states = set(accept_states)
```

```
    def transition(self, state, symbol):
        return self.transitions.get((state, symbol), None)
```

```
    def accepts(self, input_string):
        current_state = self.start_state
        for symbol in input_string:
            current_state = self.transition(current_state, symbol)
            if current_state is None:
                return False
        return current_state in self.accept_states
```

```
    def __eq__(self, other):
        if self.alphabet != other.alphabet:
            return False
```

```
        # Проверяем, что количество состояний совпадает
        if len(self.states) != len(other.states):
            return False
```

```
        # Проверяем, что количество финальных состояний совпадает
        if len(self.accept_states) != len(other.accept_states):
            return False
```

```
        # Пытаемся найти биекцию между состояниями
        states1 = list(self.states)
        states2 = list(other.states)
```

```
        for perm in permutations(states2):
            mapping = dict(zip(states1, perm))
            # Проверяем, что начальное состояние соответствует
            if mapping[self.start_state] != other.start_state:
                continue
```

```
        # Проверяем, что финальные состояния соответствуют
```

```

        if set(mapping[state] for state in self.accept_states) !=
        other.accept_states:
            continue

        # Проверяем, что переходы соответствуют
        is_isomorphic = True
        for (state, symbol), target in self.transitions.items():
            if mapping.get(state, None) is None or mapping.get(target,
            None) is None:
                is_isomorphic = False
                break
            if other.transitions.get((mapping[state], symbol), None) !=
            mapping[target]:
                is_isomorphic = False
                break

        if is_isomorphic:
            return True

    return False

#Функция генерации правильного ответа
def CorrectAnswer(x, y):
    states = ["q" + str(i) for i in range(x)]
    alphabet = [str(i) for i in range(y)]
    start_state = states[0]
    accept_states = {states[0]}
    transitions = {}
    for i in range(x):
        for j in range(y):
            k = (i * y + j) % x
            transitions[(states[i], alphabet[j])] = states[k]

    #генерируется по сути json
    return {
        "states": states,
        "alphabet": alphabet,
        "transitions": transitions,
        "start_state": start_state,
        "accept_states": list(accept_states)
    }

#Костыль, который пока что приходится использовать, так как студент не может
#написать json в консоли
def parse_dfa_string(dfa_string, y):
    try:
        parts = dfa_string.split(':')
        if len(parts) != 4:
            return None

        states = parts[0].split(',')
        start_state = parts[1]
        accept_states = parts[2].split(',')

```

```

transitions = {}

alphabet = [str(i) for i in range(y)]

for transition in parts[3].split(';'):
    if not transition:
        continue
    transition_parts = transition.split('->')
    if len(transition_parts) != 2:
        continue
    source, target = transition_parts
    source = source.strip('(').split(',')
    if len(source) != 2:
        continue
    state, symbol = source[0].strip(), source[1].strip()
    transitions[(state, symbol)] = target.strip()

return {
    "states": states,
    "alphabet": alphabet,
    "start_state": start_state,
    "accept_states": accept_states,
    "transitions": transitions
}
except Exception as e:
    return None
#Функция проверки ответа на правильность
def CheckAnswer(x, y, inserted_dfa_string):
    correct_dfa_data = CorrectAnswer1(x, y)

    inserted_dfa_data = parse_dfa_string(inserted_dfa_string,y)
    if inserted_dfa_data is None:
        return False

    correct_dfa = DFA(
        correct_dfa_data["states"],
        correct_dfa_data["alphabet"],
        correct_dfa_data["transitions"],
        correct_dfa_data["start_state"],
        correct_dfa_data["accept_states"]
    )

    inserted_dfa = DFA(
        inserted_dfa_data["states"],
        inserted_dfa_data["alphabet"],
        inserted_dfa_data["transitions"],
        inserted_dfa_data["start_state"],
        inserted_dfa_data["accept_states"]
    )

```

```

return correct_dfa == inserted_dfa

#Функция, регулирующая вызов других функций
if __name__ == "__main__":
    if len(sys.argv) < 2:
        print(json.dumps({"error": "Usage: python script.py <function_name> [args...]"}))
        sys.exit(1)

    function_name = sys.argv[1]

    if function_name == "CorrectAnswer":
        args_json = sys.argv[2]
        args = json.loads(args_json)
        x = args['x']
        y = args['y']
        result = CorrectAnswer(x, y)
        print(json.dumps(result))

    elif function_name == "CheckAnswer":
        args_json = sys.argv[2]
        args = json.loads(args_json)
        x = args['x']
        y = args['y']
        inserted_dfa_json = sys.argv[3]
        result = CheckAnswer(x, y, inserted_dfa_json)
        print(json.dumps({"result": result}))
    else:
        print(json.dumps({"error": f"Unknown function: {function_name}"}))

```